Итоговый проект · Автоматизация разработки и тестирования ML-моделей

Пайплайн PD-модели для клиентов кредитных карт

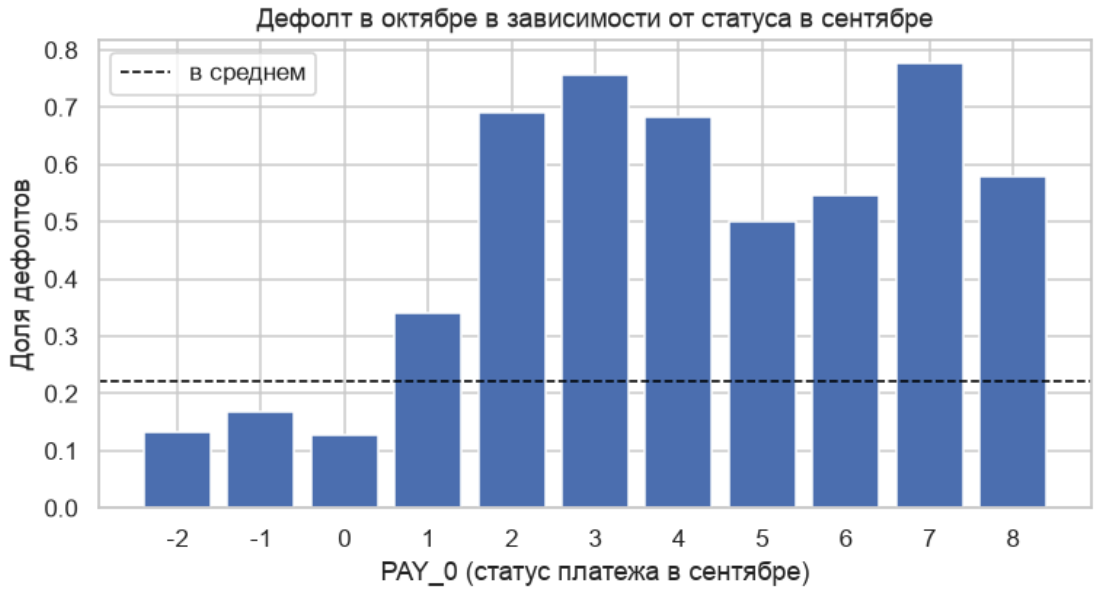
Сквозная автоматизация: от проверки входящих данных до REST API в Docker и мониторинга дрифта

Great Expectations | scikit-learn | MLflow | DVC | pytest | GitHub Actions | FastAPI | Docker | PSI

github.com/deepmindme/credit-pd-pipeline

Задача и данные

- Я в роли ML-инженера кредитного отдела: нужно автоматизировать разработку и обслуживание модели вероятности дефолта (PD).
- Решение должно быть **воспроизводимым, тестируемым и готовым к развёртыванию**.
- Датасет UCI Default of Credit Card Clients: 30 000 клиентов тайваньского банка, апрель–сентябрь 2005.
- 23 признака: лимит, демография, статусы платежей, суммы счетов и платежей за 6 месяцев.
- Таргет: дефолт в октябре. Таких клиентов **22%**.



Архитектура пайплайна

validate
Great Expectations, 78 правил; при нарушении пайплайн стоп

→

prepare
типы, дубликаты, стратифицированный сплит 80/20

→

train
6 экспериментов, CV-поиск, порог, MLflow + Registry

→

evaluate
метрики, графики, gains-таблица, эталон для PSI

DVC — `dvc.yaml` описывает стадии, данные и модели версионируются, remote — ветка `dvc-storage` по HTTP

MLflow — параметры, метрики, артефакты, модели, датасеты; вложенные runs поиска

FastAPI + Docker — `/predict`, `/predict/batch`, `/health`; лёгкий образ для инференса

Мониторинг — поток заявок в API, PSI по скорам и 23 признакам, гейт `--fail-on-drift`

CI/CD — black, flake8, GE-валидация, 81 тест, `dvc status`, сборка образа, smoke-тест

Главное решение: очистка категорий и feature engineering живут **внутри** sklearn Pipeline, поэтому API принимает исходные 23 поля, а train/serve skew исключён.

Данные: EDA и валидация

- Пропусков нет, но есть **35 дубликатов** без учёта ID. Удаляю, чтобы не было утечки между train и test.
- Недокументированные коды: `EDUCATION` 0/5/6, `MARRIAGE` 0. В suite они разрешены, в модели сведены к «другое».
- Самый сильный сигнал — история просрочек `PAY_x`. Сырые суммы счетов почти бесполезны (IV ≈ 0.01).

Great Expectations suite (78 правил):

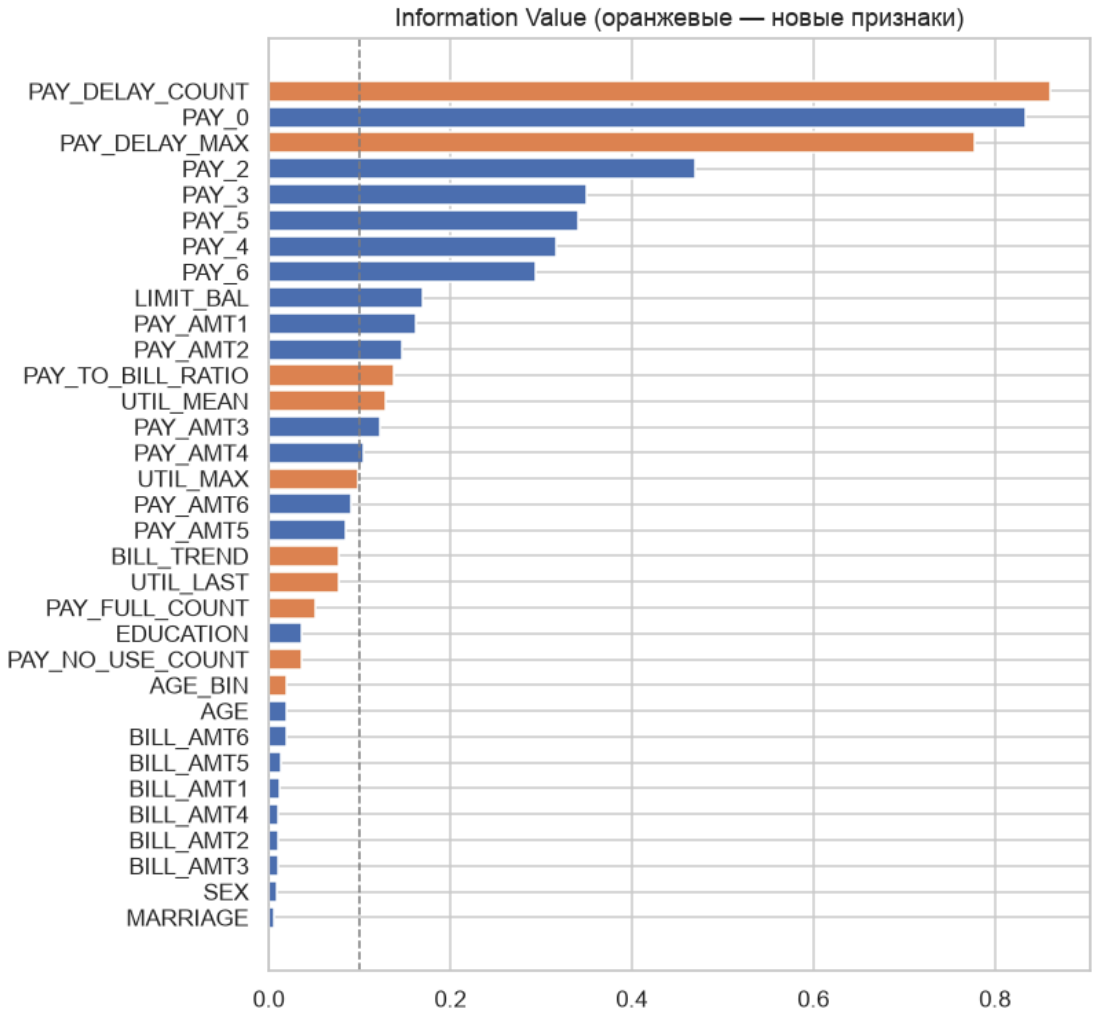
- состав колонок, null, типы, уникальность ID;
- коды категорий, диапазоны (возраст 18–100, платежи ≥ 0);
- таргет ∈ {0, 1}, доля дефолтов 5–50%.

CLI возвращает код 1 при нарушении. В тестах проверены **14 видов аномалий**, и отдельный тест падает, если аномалии появятся в реальной выгрузке.

Feature engineering

ПРИЗНАК	СМЫСЛ
`PAY_DELAY_COUNT / MAX`	месяцы с просрочкой, макс. просрочка
`PAY_NO_USE / FULL_COUNT`	не пользовался / гасил полностью
`UTIL_LAST / MEAN / MAX`	загрузка лимита
`BILL_TREND`	рост долга за полгода
`PAY_TO_BILL_RATIO`	платёж t к счёту t–1
`AGE_BIN`	U-образный риск по возрасту

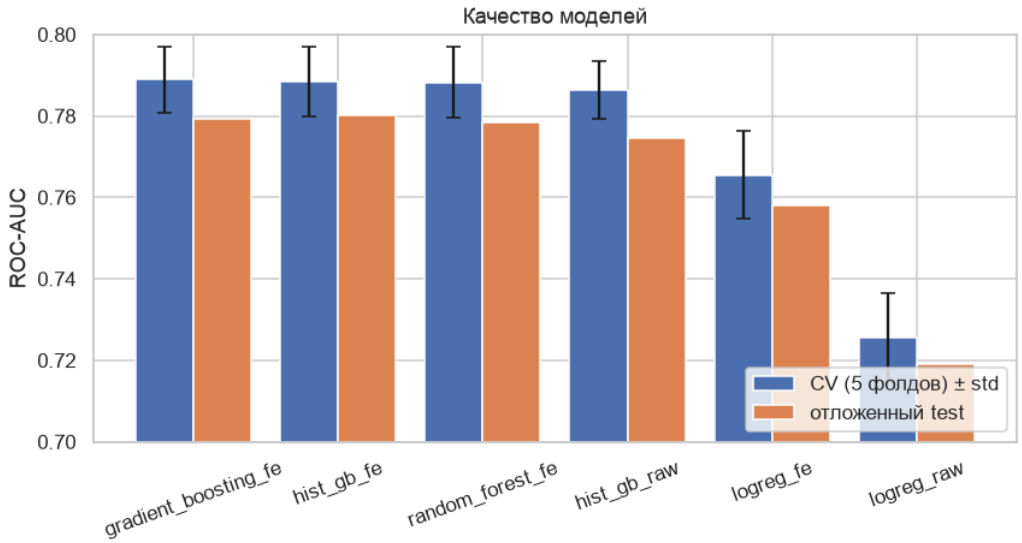
Новый `PAY_DELAY_COUNT` оказался **самым информативным** признаком: IV 0.86 против 0.83 у `PAY_0`.



Эксперименты: 6 моделей, выбор по CV

ЭКСПЕРИМЕНТ	CV AUC	TEST AUC	F1	ВРЕМЯ
logreg_raw	0.726	0.719	0.495	2 с
logreg_fe	0.766	0.758	0.529	1 с
random_forest_fe	0.788	0.778	0.539	97 с
gradient_boosting_fe	**0.789**	**0.779**	**0.536**	**93 с**
hist_gb_raw	0.786	0.774	0.527	7 с
hist_gb_fe	0.789	0.780	0.540	8 с

- FE: **+0.040** AUC логрегрессии и лишь +0.002 бустингу.
- Ансамбли различаются меньше, чем на 1 std CV (≈ 0.008).
- Test для выбора модели не используется.



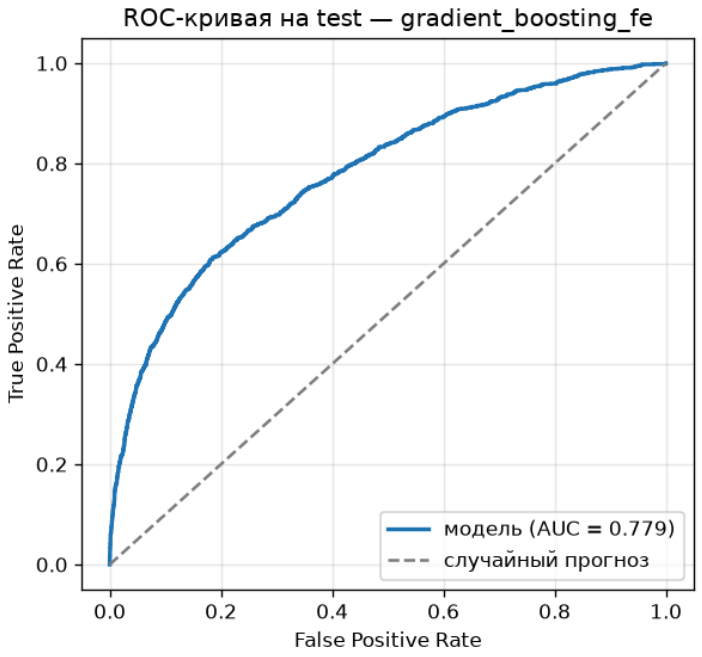
MLflow Tracking и Registry

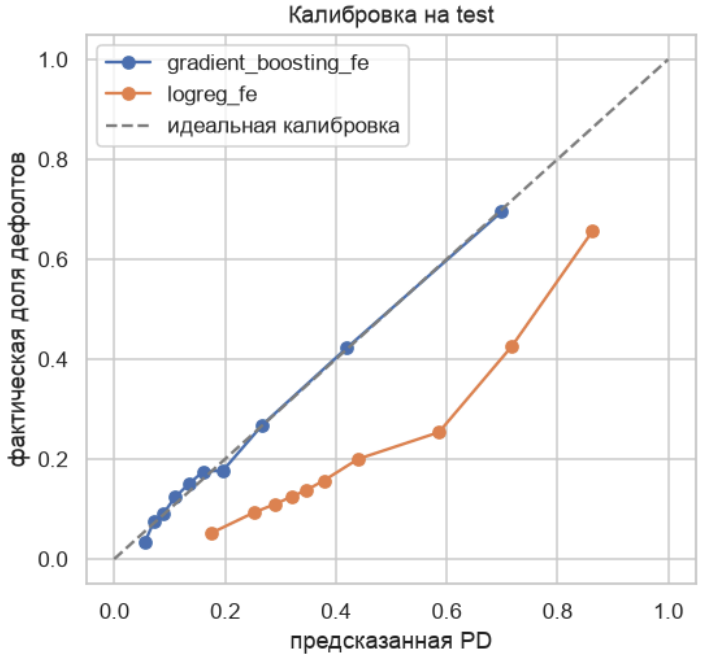
						Metrics				Parameters
	Run Name	Created	Duration	Models	Description	cv_roc_auc_m	test_f1	test_gini	test_roc_auc	decision_thres
☐	hist_gb_fe	8 minutes ago	10.7s	model	HistGradientBoosting + feature engineering	0.7884643...	0.5403533...	0.5603517...	0.7801758...	0.2653
☐	hist_gb_raw	8 minutes ago	10.6s	model	HistGradientBoosting на исходных признаках	0.7863026...	0.5270175...	0.5488604...	0.7744302...	0.2673
☐	gradient_boosting_fe	10 minutes ago	1.6min	credit-default-pd v1	GradientBoostingClassifier + feature engineering	0.7889229...	0.5362844...	0.5587070...	0.7793535...	0.2767
☐	random_forest_fe	11 minutes ago	1.7min	model	Случайный лес + feature engineering	0.7882672...	0.5390738...	0.5567272...	0.7783636...	0.2932
☐	logreg_fe	11 minutes ago	3.3s	model	Логистическая регрессия + агрегаты истор...	0.7654529...	0.5284640...	0.5161105...	0.7580552...	0.5938
☐	logreg_raw	12 minutes ago	4.5s	model	Бейзлайн: логистическая регрессия на исхо...	0.7256160...	0.4950980...	0.4382734...	0.7191367...	0.5841

- **Параметры**: лучшие гиперпараметры, тип поиска, порог.
- **Метрики**: CV AUC ± std, test AUC/Gini/KS/P/R/F1/Brier.
- **Артефакты**: ROC, PR, матрица ошибок, importance, `cv_results`.
- **Модель** с сигнатурой и кодом; **86 вложенных runs** поиска.
- Лучшая модель зарегистрирована как `credit-default-pd` с алиасом `champion`.

Итоговая модель: GradientBoosting + FE

0.779	0.559	0.428	0.517	0.557	0.536
ROC-AUC на test	Gini	KS	Precision	Recall	F1





- Порог **0.277** подобран по F1 на CV по train. При пороге 0.5 нашлось бы только 35% дефолтов.
- PD **откалибрована**: по децилям расхождение с фактом не больше 2 п.п.
- Логрегрессия с `balanced` завышает риск **в 2–3 раза**.
- Топ-10% по PD содержат 31% дефолтов.

DVC и CI/CD

```
$ dvc repro
validate → prepare → train → evaluate

$ dvc metrics show
reports/metrics.json  roc_auc 0.7794  gini 0.5587
                      f1 0.5363  threshold 0.2767
```

- Под DVC: датасет, train/test, модель, эталон для PSI.
- Remote — ветка `dvc-storage` по HTTP: `dvc pull` без ключей.
- Параметры собраны в `params.yaml`.

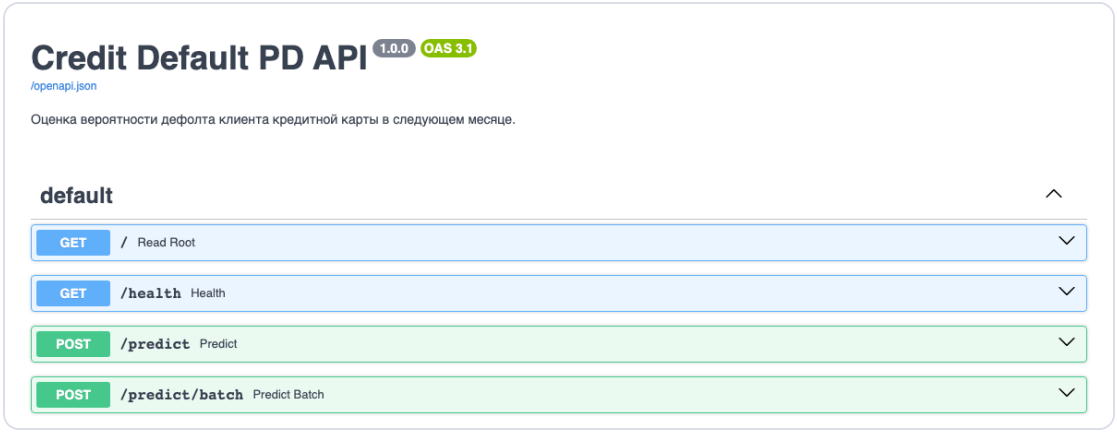
GitHub Actions, на каждый push:

- **lint**: black, flake8;
- **test**: `dvc pull` → **валидация GE** → 81 тест pytest;
- **dvc**: `dvc.lock` соответствует коду, стадии воспроизводятся байт в байт;
- **docker**: сборка образа, запуск, smoke-тест `/predict`, мониторинг PSI.

REST API и Docker

```
$ ./scripts/docker_build_run.sh   # сборка, запуск,
проверка /health

$ curl -X POST localhost:8000/predict \
    -H "Content-Type: application/json" \
    -d @scripts/sample_request.json
{"default_prediction": 1,
 "default_probability": 0.621562,
 "threshold": 0.276678}
```



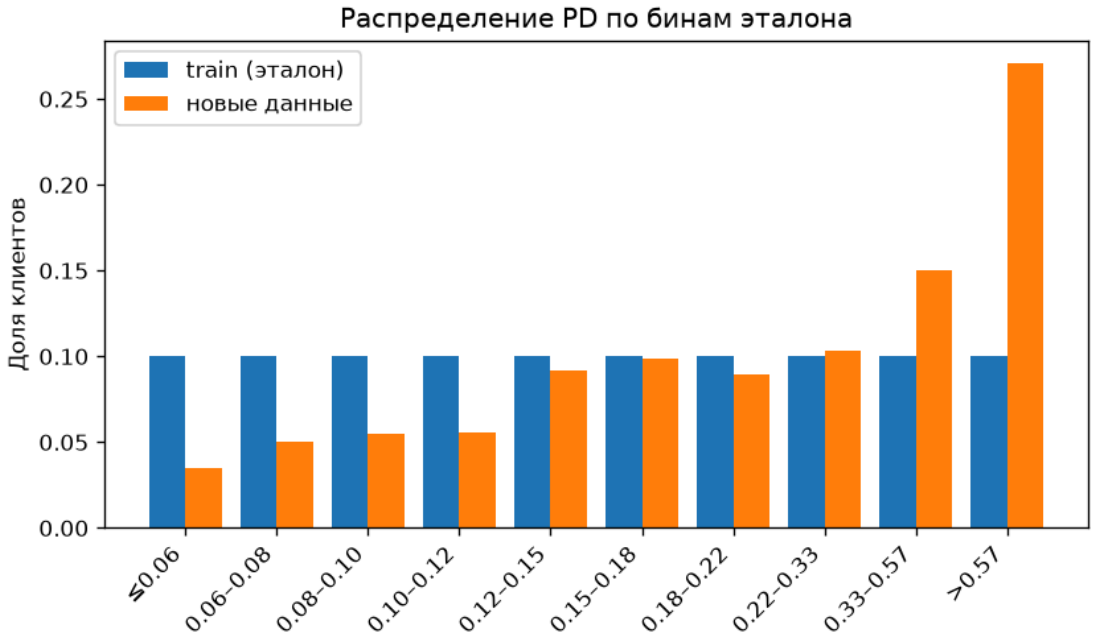
- `POST /predict`: клиент → класс и PD.
- `POST /predict/batch`: до 1000 клиентов.
- `GET /health`: версия модели и порог.
- Pydantic-схема повторяет правила GE: возраст 12 или `PAY_0 = 11` → 422.
- Образ `python:3.11-slim` только с зависимостями инференса, без root, с `HEALTHCHECK`.
- Без модели сервис не падает: `/health` и `/predict` отвечают 503.
- Проверено локально (контейнер *healthy*, ответ выше) и в CI на каждый push.

Мониторинг: Population Stability Index

- Скрипт берёт 2000 клиентов из test и отправляет их в API.
- PSI считается по распределению PD и по каждому из 23 признаков; эталон строится по train.

СЦЕНАРИЙ	PSI СКОРОВ	СТАТУС
baseline	0.003	stable
drift моложе, лимиты –40%, больше просрочек	0.348	**significant**

В drift-сценарии сработали `AGE`, `LIMIT_BAL`, `PAY_0…PAY_6`. В CI это гейт: код 2 при PSI ≥ 0.25.



Итоги и что дальше

Сделано

- Воспроизводимый пайплайн одной командой `dvc repro`.
- Данные проверяются до обучения и в каждом CI-прогоне.
- 6 экспериментов в MLflow; модель откалибрована, ROC-AUC 0.779.
- API в Docker, 81 тест, мониторинг дрифта с гейтом.

Что улучшить

- Подбирать порог по стоимости ошибок, а не по F1.
- Убрать пол из признаков (требования недискриминации).
- Вынести MLflow на сервер, DVC remote — в S3.
- Out-of-time валидация на данных за другие периоды.
- Перейти на `hist_gb_fe`: то же качество в 12 раз быстрее.